

Design Patterns

Week 2/3

Behavioral Patterns

10 PATTERNS

Complete collection

DESI EXAMPLES

Easy samajh aaye

Objects ka behavior samjho →

1 Observer Pattern

EXAMPLE

YouTube - Subscribe karo, notification milta hai. One YouTuber, many subscribers.

PROBLEM

One object changes, many need updates. Hard-coding dependencies is messy.

SOLUTION

Subject keeps observer list. When change happens, notify all automatically.

CODE

```
interface Observer {  
    void update(String msg);  
}  
  
class Channel {  
    List<Observer> subs;  
    void notify() {  
        for(Observer o : subs)  
            o.update("New video");  
    }  
}
```

USE CASE

→ YouTube notifications

→ Stock price updates

→ Event listeners in UI

2 Strategy Pattern

EXAMPLE

Paytm payment - UPI/Card/Wallet. Same kaam, alag tareeka. Runtime pe choose karo.

PROBLEM

Multiple ways to do one task. If-else mess everywhere.

SOLUTION

Make algorithms interchangeable. Switch at runtime without code change.

CODE

```
interface Payment {  
    void pay(int amt);  
}  
  
class UPI implements Payment {  
    void pay(int amt) {...}  
}  
  
cart.setPayment(new UPI());  
cart.checkout();
```

USE CASE

→ Payment methods (UPI/Card)

→ Sorting (Quick/Merge)

→ Google Maps routes

3 Command Pattern

EXAMPLE

Swiggy order - Order object ban gaya. Store, cancel, track kar sakte ho.

PROBLEM

Need undo/redo, queue actions, log operations. Direct calls cant do this.

SOLUTION

Wrap request in command object. Can store, queue, undo commands.

CODE

```
interface Command {  
    void execute();  
    void undo();  
}  
  
class OrderFood  
    implements Command {  
    void execute() {...}  
    void undo() {...}  
}
```

USE CASE

→ Swiggy orders

→ Undo/Redo (Ctrl+Z)

→ Remote control buttons

4 Template Method

EXAMPLE

Chai banana - Steps same (boil, add, brew). Ingredients vary (masala, ginger).

PROBLEM

Algorithm skeleton same, some steps differ. Copy-paste everywhere.

SOLUTION

Define skeleton in base class. Subclasses override specific steps only.

CODE

```
abstract class Chai {  
    void make() {  
        boilWater();  
        addIngredients();  
        brew();  
    }  
    abstract void  
        addIngredients();  
}
```

USE CASE

→ Recipe variations

→ Data pipelines

→ Test setup/teardown

5

State Pattern

EXAMPLE

OLA ride - Searching, Assigned, Riding states. Cancel button behaves differently.

PROBLEM

Object behavior changes with state. Too many if-else checks.

SOLUTION

Create state objects. Delegate behavior to current state object.

CODE

```
interface State {  
    void cancel();  
}  
  
class Riding  
    implements State {  
    void cancel() {  
        // Charge fee  
    }  
}
```

USE CASE

→ OLA/Uber rides

→ Vending machines

→ Order status tracking

6 Chain of Responsibility

EXAMPLE

Customer care escalation - L1 support nahi kar paya, L2 ko pass. L2 nahi, manager ko.

PROBLEM

Multiple handlers can process request. Dont know which one will handle.

SOLUTION

Chain handlers. Each decides to handle or pass to next in chain.

CODE

```
abstract class Handler {
    Handler next;
    void handle(Request r) {
        if (canHandle(r))
            process(r);
        else if (next != null)
            next.handle(r);
    }
}
```

USE CASE

→ Customer support escalation

→ Logging levels

→ Authentication filters

7

Iterator Pattern

EXAMPLE

Playlist songs - Next song button. Collection internal structure hide karo, iterate karne do.

PROBLEM

Want to traverse collection without exposing internal structure.

SOLUTION

Provide iterator object. Client uses hasNext() and next() methods.

CODE

```
interface Iterator {  
    boolean hasNext();  
    Object next();  
}  
  
class Playlist {  
    Iterator getIterator() {  
        return new  
            SongIterator();  
    }  
}
```

USE CASE

→ Playlist navigation

→ Java Collections

→ File system traversal

8 Mediator Pattern

EXAMPLE

WhatsApp group - Sabko individually message mat karo. Group pe bhejo, sab ko mil jayega.

PROBLEM

Objects talk directly to each other. Tight coupling, messy dependencies.

SOLUTION

Central mediator handles communication. Objects talk via mediator only.

CODE

```
class ChatRoom {  
    void sendMsg(User from,  
                String msg) {  
        for (User u : users)  
            u.receive(msg);  
    }  
}  
  
user.send("Hello", room);
```

USE CASE

→ Chat applications

→ Air traffic control

→ UI form validation

9 Memento Pattern

EXAMPLE

Game save/load - Progress save karo, baad mein wahi se continue. State restore ho jata.

PROBLEM

Need to save and restore object state without breaking encapsulation.

SOLUTION

Memento stores snapshot. Originator creates and restores from memento.

CODE

```
class Memento {  
    private State state;  
}  
  
class Game {  
    Memento save() {  
        return new  
            Memento(state);  
    }  
    void restore(Memento m)  
}
```

USE CASE

→ Game save points

→ Document version history

→ Database rollback

10

Visitor Pattern

EXAMPLE

Income tax officer - Ghar ghar jaake different rules apply karta. Salaried, business, freelancer alag.

PROBLEM

Add new operations to objects without modifying their classes.

SOLUTION

Visitor object visits elements. Each element accepts visitor.

CODE

```
interface Visitor {  
    void visit(Salaried s);  
    void visit(Business b);  
}  
  
class TaxCalc  
    implements Visitor {  
    void visit(Salaried s) {  
        // Calculate tax  
    }  
}
```

USE CASE

→ Tax calculations

→ File system operations

→ Compilers (AST traversal)

10 Behavioral Patterns

01 **Observer**
One-to-many notifications

02 **Strategy**
Switch algorithms at runtime

03 **Command**
Action as object (undo/redo)

04 **Template**
Fixed steps, flexible parts

05 **State**
Behavior changes with state

06 **Chain**
Pass request along chain

07 **Iterator**
Traverse without exposing

08 **Mediator**
Central communication hub

09 **Memento**
Save and restore state

10 **Visitor**
Add operations to objects

Master karo interviews ke liye!